

Week 6 · 실습 워크북: 콘솔이 개입한 뒤에 무엇을 어느 쪽으로 맞출 것인가

📌 참고

이 문서는 실습 파트(45분)입니다. 개념은 같은 `lecture/` 폴더의 **개념워크북** 을 먼저 보세요. 여기서는 번호만 가리킵니다. **[실습]** 은 직접 실행하고, **[확인]** 은 화면을 대조하고, **[함정]** 은 미리 피하고, **[관찰 🕒]** 은 빈칸을 채웁니다. **(심화 ☆)** 는 먼저 끝낸 사람만 합니다.

작업 폴더는 `practice/` 입니다. `solution/` 은 막혔을 때 답을 보는 곳입니다. 위에서 아래로 그대로 따라가면 끝납니다.

오늘 만들 것

S3 버킷 두 개뿐입니다. 그런데 둘이 생기는 방식이 정반대라서, 같은 `plan` 화면을 두 가지 방향으로 읽게 됩니다.

버킷	누가 먼저 만드나	오늘 하는 일	Block
<code>{project_name}-drift</code>	코드가 먼저. Terraform 이 만듭니다	콘솔에서 태그를 직접 바꿔 드리프트를 만들고, <code>plan</code> 으로 확인합니다	A
<code>{project_name}-manual</code>	콘솔이 먼저. 직접 만듭니다	<code>terraform import</code> 로 state 에 편입하고, 코드를 채웁니다	B

`terraform` 명령을 치는 곳은 `practice/` 한 곳입니다. 브라우저의 AWS 콘솔도 함께 열어두세요. 오늘은 터미널과 콘솔을 번갈아 봅니다.

🚫 절대 금지

비용. 빈 S3 버킷은 요금이 붙지 않습니다. 이번 주는 EC2 가 없어서 시간당 요금도 없습니다. 그래도 Block C 는 필수입니다. S3 버킷 이름은 전 세계에서 유일해서 남겨두면 그 이름을 아무도 못 쓰고, **import 한 버킷도 `destroy` 대상이 된다는 것을 확인하는 것이 이번 주 마무리입니다.** 개념워크북 17번.

버킷 안에 파일을 올리지 마세요. 객체가 있으면 `destroy` 가 `BucketNotEmpty` 로 실패합니다.

올리면 안 되는 값. AWS 계정번호 12자리 · 액세스 키 · `terraform.tfvars` · `terraform.tfstate` . 마스크 명령은 C-3에 있습니다.

🚫 절대 금지

콘솔에서 건드려도 되는 것은 딱 둘입니다. drift 버킷의 `Env` 태그를 바꾸는 것과, manual 버킷을 새로 만드는 것. 그 밖의 리소스는 손대지 마세요. 특히 지난 주차에서 만든 week3 의 state 버킷(`*-tfstate`)과 잠금 테이블(`*-tflock`)은 절대 지우면 안 됩니다. 개념워크북 6번.

! 꼭 기억

이 문서의 코드블록 첫 줄에 붙은 `# pwd == ...` 주석은 그 블록의 명령을 다 친 뒤에 있어야 하는 위치입니다. 블록 안에 `cd` 가 있으면 시작 위치는 그 위 블록의 값입니다. 매 블록 끝에 `pwd` 로 대조하세요.

파일	오늘 손대나
<code>practice/versions.tf</code> · <code>providers.tf</code> · <code>variables.tf</code>	읽기만
<code>practice/backend.tf</code>	TODO ① week3 버킷 이름을 직접 입력
<code>practice/main.tf</code>	TODO ②③④ A-3 에서 하나, B-2 에서 하나, B-4 에서 둘
<code>practice/outputs.tf</code>	TODO ⑤ import 한 버킷을 output 으로
<code>practice/example.tfvars</code>	<code>terraform.tfvars</code> 로 복사해 값 채우기

오늘의 지도: 세 곳에 적힌 같은 값과, 그것을 맞추는 두 방향

week1의 지도는 명령의 흐름, week2는 리소스 사이의 참조, week3은 state 파일이 사는 곳, week4는 값이 어디에 적혀 있는지, week5는 코드가 어느 폴더에 있는지였습니다. 오늘의 지도는 같은 값이 세 곳에 적혀 있다는 것과, 그 셋이 어긋났을 때 화살표가 어느 방향으로 움직이는지입니다.

① 세 곳: 코드 · state · 실물

코드	state	실물
<code>practice/main.tf</code>	S3 의 <code>week06/app/terraform.tfstate</code>	AWS 계정의 버킷
<code>Env = "study"</code>	<code>"Env": "study"</code>	<code>Env : study</code>
내가 직접 씁니다 git 에 올라갑니다 terraform plan 이 읽습니다	apply 가 씁니다 S3 에 올라갑니다 terraform state show 가 읽습니다	apply 가 만듭니다 콘솔에서 눈으로 봅니다 AWS 콘솔이 보여줍니다
셋이 같으면 plan 이 No changes 라고 말합니다.		

② Block A: 콘솔이 나중에 끼어듭니다

- 1) apply 직후
코드 study = state study = 실물 study
|
| 사람이 손댔
- 2) 콘솔에서 Env 를 HACKED 로
코드 study = state study ≠ 실물 HACKED
^
state 는 아직 옛날 값입니다.

terraform state show 로 확인합니다. (A-4)

3) terraform plan refresh 가 실물을 물어보고 화면에 차이를 보여줍니다
Plan: 0 to add, 1 to change, 0 to destroy. (A-5)

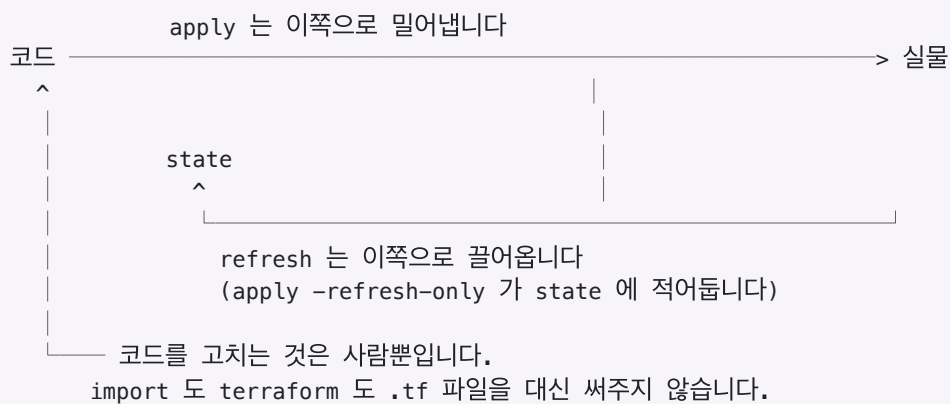
4) 여기서 길이 둘로 갈립니다. (A-6)

terraform apply -refresh-only	terraform apply
state 만 실물에 맞춥니다	실물을 코드에 맞춥니다
실물은 그대로 HACKED	실물이 study 로 돌아옵니다
코드는 그대로 study	셋이 다시 갈아집니다
-> plan 은 여전히 1 to change	

③ Block B: 콘솔이 먼저 만들어 놓습니다

- | | | | |
|---------------------|---|----------|---|
| 1) 콘솔에서 버킷 생성 | 코드 없음 | state 없음 | 실물 있음 (버전 관리 ON) |
| | | | ^
Terraform 은 존재 자체를 모릅니다.
plan 에 나오지 않습니다. (B-1) |
| 2) 코드에 빈 블록만 | 코드 있음 | state 없음 | 실물 있음 |
| | plan 은 "새로 만들겠다"고 합니다. 여기서 apply 하면 안 됩니다. | | |
| 3) terraform import | 코드 있음 | state 있음 | 실물 있음 (B-2) |
| | import 는 state 만 채웁니다. 코드는 안 채웁니다. | | |
| 4) terraform plan | No changes. | | (B-3) |
| | 그런데 버전 관리는 코드 어디에도 없습니다.
plan 은 코드에 적힌 것만 비교합니다. | | |
| 5) 부속 리소스 블록 2개 추가 | plan: 2 to add. 실물에 이미 있는데도 그렇습니다. (B-4) | | |
| 6) 각각 import | plan: No changes. state list 4줄 | | (B-5) |

④ 두 명령이 미치는 방향



세 번째 줄이 오늘 겪을 불편의 자리입니다. state 는 명령으로 채워지는데 코드는 사람이 씁니다.

스텝 ↔ 지도 대응표

스텝	지도에서 하는 일
A-3	①의 세 곳이 모두 <code>study</code> 인 상태를 만듭니다
A-4	②의 2번. <code>state</code> 가 옛날 값인 것을 눈으로 봅니다
A-5	②의 3번. <code>plan</code> 화면 두 덩어리를 읽습니다
A-6	②의 4번. 두 길을 모두 걸어봅니다
B-1	③의 1번. Terraform 이 모르는 리소스를 만듭니다
B-2	③의 2 · 3번. 빈 블록을 적고 <code>import</code> 합니다
B-3	③의 4번. <code>No changes</code> 의 뜻을 확인합니다
B-4	③의 5번. 부속 리소스가 따로라는 것을 봅니다
B-5	③의 6번. ④의 세 번째 줄을 몸으로 겪습니다
C-1	<code>import</code> 한 것도 지워집니다

오늘 나와야 하는 숫자

이 표와 다르면 멈추고 물어보세요. 넘어가면 뒤에서 더 크게 터집니다.

지점	나와야 하는 값	다르면
A-1 버킷 확인	OK: 버킷 있음 과 <code>ACTIVE</code>	week3 bootstrap 확인
A-3 <code>init</code>	<code>Terraform has been successfully initialized!</code>	버킷 이름 확인
A-3 <code>plan</code>	<code>Plan: 1 to add, 0 to change, 0 to destroy.</code>	즉시 멈춤. 아래 WARNING
A-3 <code>state list</code>	1줄 (<code>aws_s3_bucket.drift_demo</code>)	멈춤
A-4 <code>state show</code> 의 <code>Env</code>	아직 <code>"study"</code>	콘솔 저장에 안 됐거나 다른 버킷을 고쳤습니다
A-5 <code>plan</code>	<code>Plan: 0 to add, 1 to change, 0 to destroy.</code>	콘솔 변경이 저장됐는지 확인
A-6 <code>-refresh-only</code> 후 <code>state show</code>	<code>"HACKED"</code> 로 바뀜	멈춤
A-6 <code>-refresh-only</code> 후 <code>plan</code>	여전히 <code>1 to change</code>	개념워크북 8번 다시 읽기

지점	나와야 하는 값	다르면
A-6 <code>apply</code> 후 <code>plan</code>	<code>No changes.</code>	멈춤
B-2 <code>import</code>	<code>Import successful!</code>	버킷 이름과 리전 확인
B-2 <code>state list</code>	2줄	멈춤
B-3 <code>plan</code>	<code>No changes.</code>	콘솔에서 태그를 붙였다면 그 diff 가 나옵니다
B-4 <code>plan</code>	<code>Plan: 2 to add, 0 to change, 0 to destroy.</code>	여기서 <code>apply</code> 하지 마세요
B-5 <code>plan</code>	<code>No changes.</code>	B-5 의 <code>import</code> 두 개를 다 했는지 확인
B-5 <code>state list</code>	4줄	멈춤
B-5 S3 객체	<code>week06/app/terraform.tfstate</code> 1개	<code>backend.tf</code> 의 <code>key</code> 확인
C-1 <code>destroy</code>	<code>Destroy complete! Resources: 4 destroyed.</code>	멈춤
C-2 <code>state list</code>	빈 출력	멈춤

⚠ 주의

A-3의 `1 to add` 가 오늘의 안전 게이트입니다. `backend.tf` 의 `key` 를 지난주 값인 `week05/prod/...` 로 잘못 두면 week6 스택이 week5 의 state 를 인수합니다. 그 state 에는 VPC 와 EC2 가 들어 있으니 `plan` 에 삭제나 변경이 섞여 나옵니다. 그대로 `apply` 하면 지난주 리소스를 건드립니다.

숫자가 1이 아니거나 `-` 가 섞여 있으면 `apply` 하지 말고 멈추세요. `backend` 블록에는 변수도 검사도 넣을 수 없어서 코드로 막을 방법이 없습니다. 이 확인이 유일한 방어선입니다. week5 개념워크북 13번.

⚠ 주의

지난 주차들과 달리 오늘은 `plan` 의 숫자가 실습 도중 여러 번 바뀝니다. `1 to add` → `1 to change` → `No changes` → `2 to add` → `No changes` 순서입니다. 숫자 하나만 외우지 말고 **지금 어느 스텝에 있는지와 함께** 위 표를 보세요.

Block A. 워크스루 (18분): 콘솔이 나중에 끼어듭니다

강사와 함께 진행합니다. 오늘 처음 나오는 명령이 셋(`state show` · `apply -refresh-only` · `import`)이라 손이 익을 때까지 같이 칩니다.

[확인] A-1. 사전 점검 (2분)

지금 하는 일은 도구와 week3의 저장소가 살아 있는지 보는 것입니다. {본인-github-id} 자리는 셀이 채워지지 않으니 week3에서 쓴 `project_name` 을 직접 넣으세요.

```
# pwd == week6-import-drift
terraform version          # v1.15.8 이상
aws sts get-caller-identity # 계정번호와 사용자 확인

aws s3api head-bucket --bucket "boaz26-w3-{본인-github-id}-tfstate" >/dev/null 2>&1 \
  && echo "OK: 버킷 있음"
aws dynamodb describe-table --region ap-northeast-2 \
  --table-name "boaz26-w3-{본인-github-id}-tflock" --query 'Table.TableStatus' --output text
```

OK: 버킷 있음
ACTIVE

! 꼭 기억

둘 중 하나라도 실패하면 여기서 멈추고 강사에게 알려주세요. week3의 `bootstrap` 스택을 다시 apply 해야 합니다. 그때 `project_name` 은 week3에서 쓴 값과 글자 하나까지 같아야 합니다. 버킷 이름이 거기에 걸려 있습니다.

출력된 계정번호 12자리는 화면 공유나 스크린샷에 넣지 마세요.

브라우저에서 AWS 콘솔을 열고 **리전이 서울(ap-northeast-2)인지** 확인하세요. 오른쪽 위에 리전 이름이 나옵니다. 오늘 콘솔과 터미널을 번갈아 쓰는데, 리전이 다르면 콘솔에서 만든 것을 터미널이 못 찾습니다.

파일도 눈으로 확인합니다.

```
# pwd == week6-import-drift
ls -l practice/
```

```
backend.tf
example.tfvars
main.tf
outputs.tf
providers.tf
README.md
variables.tf
versions.tf
```

week5와 달리 폴더가 하나이고 `modules/` 가 없습니다. 오늘은 모듈을 쓰지 않습니다. `module.` 접두사가 붙으면 import 주소가 길어져서 초점이 흐려지기 때문입니다. 개념워크북 15번.

[실습] A-2. TODO ①: week3 버킷을 이어 쓰고 값 채우기 (3분)

지금 하는 일은 오늘 만들 것의 state 를 week3 버킷의 새 `key` 에 두는 것입니다.

```
# pwd == practice
cd practice
cp example.tfvars terraform.tfvars
```

`terraform.tfvars` 를 열어 `project_name` 을 본인 값으로 바꿉니다. `region` 은 그대로 둡니다.

```
region      = "ap-northeast-2"
project_name = "boaz26-w6-kdh1834"
```

⚠ 주의

`project_name` 은 그대로 S3 버킷 이름이 됩니다. 버킷 이름은 AWS 계정 안이 아니라 전 세계에서 유일해야 합니다. 다른 사람이 이미 쓰고 있는 이름이면 만들 때 실패합니다. 본인 github id 를 꼭 넣으세요.

소문자 · 숫자 · 하이픈만 쓸 수 있습니다. 대문자나 밑줄을 넣으면 `variables.tf` 의 검사에 걸립니다.

```
Error: Invalid value for variable

on terraform.tfvars line 1:
 1: project_name = "B0AZ26-W6-KDH1834"
   |
   | var.project_name is "B0AZ26-W6-KDH1834"
```

S3 버킷 이름 규칙입니다. 소문자 · 숫자 · 하이픈만 쓸 수 있습니다.

이제 `backend.tf` 를 열어 `CHANGE-ME` 두 곳을 week3 값으로 바꿉니다. 방금 정한 `project_name` 과 다른 값입니다.

```
terraform {
  backend "s3" {
    bucket      = "boaz26-w3-kdh1834-tfstate"
    key         = "week06/app/terraform.tfstate"
    region     = "ap-northeast-2"
    dynamodb_table = "boaz26-w3-kdh1834-tflock"
    encrypt     = true
  }
}
```

⚠ 주의

`key` 는 이미 채워져 있습니다. 건드리지 마세요. week5 코드에서 복사해 오면 `key` 가 `week05/prod/terraform.tfstate` 로 나옵니다. 그 줄까지 붙여넣으면 지난주 state 를 인수해서 VPC 와 EC2 가 삭제 대상이 됩니다.

`terraform.tfvars` 는 커밋하지 마세요. `.gitignore` 가 막고 있지만 확인은 본인이 합니다.

[실습] A-3. TODO ②: 드리프트 실습용 버킷 만들기 (4분)

지금 하는 일은 코드 · state · 실물 셋이 모두 같은 상태를 만드는 것입니다. 여기서부터 어긋내기 시작합니다.

`practice/main.tf` 를 열어 TODO ② 자리에 리소스 하나를 적습니다.

```
resource "aws_s3_bucket" "drift_demo" {
  bucket = "${var.project_name}-drift"

  tags = {
    Name     = "${var.project_name}-drift"
    Study    = "boaz-terraform-26"
    Env      = "study"
  }
}
```

Env 태그가 오늘의 실험 대상입니다. 태그는 무과금 속성이라 콘솔에서 바뀌도 요금이 달라지지 않습니다. 개념워크북 6번.

```
# pwd == practice
terraform init
terraform plan
```

Plan: 1 to add, 0 to change, 0 to destroy.

[함정] `Error: Failed to get existing workspaces: S3 bucket "... does not exist.` 가 나오면 버킷 이름이 틀렸거나 week3의 bootstrap 이 없는 것입니다. A-1로 돌아가세요.

[함정] `dynamodb_table` 이 deprecated 라는 경고가 함께 뜹니다. week3·4·5에서 본 그 경고이고 정상입니다.

⚠ 주의

1이 아니면 apply 하지 말고 멈추세요. 삭제나 변경이 섞여 있으면 `backend.tf` 의 `key` 가 지난주 값입니다.

```
# pwd == practice
terraform apply      # yes
terraform state list
```

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

`aws_s3_bucket.drift_demo`

지금 셋이 모두 `study` 입니다. 지도 ①의 상태입니다.

[실습] [관찰 🍷] A-4. 콘솔에서 손대고, state 를 열어봅니다 (3분)

지금 하는 일은 드리프트를 일부러 만들고, **state** 가 **자동으로 갱신되지 않는다**는 것을 눈으로 보는 것입니다.

브라우저에서 AWS 콘솔을 엽니다.

1. S3 서비스로 갑니다
2. `{project_name}-drift` 버킷을 클릭합니다
3. 위쪽 탭에서 **Properties** 를 누릅니다
4. 아래로 내려 **Tags** 항목의 **Edit** 를 누릅니다
5. **Env** 의 값을 `study` 에서 `HACKED` 로 바꿉니다
6. **Save changes** 를 누릅니다

🚫 절대 금지

태그만 바꾸세요. 같은 화면에 버킷 버전 관리 · 암호화 · 삭제 같은 항목이 있는데 손대지 마세요. 버킷을 지우면 실습이 처음부터입니다.

이제 터미널로 돌아옵니다. `plan` 을 치기 전에 state 부터 봅니다.

```
# pwd == practice
terraform state show aws_s3_bucket.drift_demo
```

출력에서 `tags` 부분을 보세요.

```
tags              = {
  "Env"   = "study"
  "Name"  = "boaz26-w6-kdh1834-drift"
  "Study" = "boaz-terraform-26"
}
```

콘솔에서는 분명히 `HACKED` 로 바꿨는데 여기는 `study` 입니다. `terraform state show` 는 **AWS 에 묻지 않고 state 파일에 적힌 것을 그대로 읽어주는 명령**이기 때문입니다. state 는 실물의 복사본이고, 누가 실물을 바꿔도 저절로 따라가지 않습니다. 개념워크북 2번.

A-4 기록

- 콘솔에서 바꾼 태그의 이름과 값: _____
- `state show` 의 `Env` 값: _____
- 콘솔의 값과 state 의 값이 다른 이유를 한 문장으로: _____

[확인] [관찰 🍷] A-5. `plan` 화면 두 덩어리를 읽습니다 (3분)

지금 하는 일은 드리프트를 감지하고, `plan` 출력에서 과거형과 미래형을 구분하는 것입니다.

```
# pwd == practice
```

```
terraform plan
```

화면이 두 덩어리로 나옵니다. 첫째 덩어리는 **이미 벌어진 일**입니다.

Note: Objects have changed outside of Terraform

Terraform detected the following changes made outside of Terraform since the last "terraform apply" which may have affected this plan:

```
# aws_s3_bucket.drift_demo has changed
~ resource "aws_s3_bucket" "drift_demo" {
  ~ tags = {
    ~ "Env" = "study" -> "HACKED"
  }
}
```

둘째 덩어리는 **Terraform 이 하겠다는 일**입니다. 화살표 방향이 반대인 것을 보세요.

Terraform will perform the following actions:

```
# aws_s3_bucket.drift_demo will be updated in-place
~ resource "aws_s3_bucket" "drift_demo" {
  ~ tags = {
    ~ "Env" = "HACKED" -> "study"
  }
}
```

Plan: 0 to add, 1 to change, 0 to destroy.

위쪽은 **has changed** (과거형), 아래쪽은 **will be updated** (미래형)입니다. 위쪽은 콘솔에서 벌어진 일이고 아래쪽은 그것을 되돌리겠다는 계획입니다. **Terraform 은 코드를 정답으로 봅니다.** 개념워크북 5번.

~ 는 지우지 않고 그 자리에서 고친다는 뜻입니다. 태그는 만든 뒤에도 바꿀 수 있는 속성이라 버킷이 새로 만들어지지 않습니다. 기호 네 개의 뜻은 개념워크북 4번에 있습니다.

A-5 기록

- plan 마지막 줄: Plan: ____ to add, ____ to change, ____ to destroy.
- 첫째 덩어리의 화살표: "Env" = ____ -> ____
- 둘째 덩어리의 화살표: "Env" = ____ -> ____
- 두 화살표의 방향이 반대인 이유를 한 문장으로: _____
- 이 줄 앞의 기호 ~ 의 뜻: _____

[실습] [관찰 🍷] A-6. 두 길을 모두 걸어봅니다 (3분)

지금 하는 일은 어긋남을 맞추는 두 방향을 각각 실행해서 무엇이 달라지는지 보는 것입니다.

먼저 **state** 를 **실물**에 맞추는 쪽입니다.

```
# pwd == practice
terraform apply -refresh-only
```

승인 문구가 평소와 다릅니다. 인프라를 바꾸지 않고 state 만 쓰겠다고 적혀 있습니다. **yes** 를 입력하세요.

```
Would you like to update the Terraform state to reflect these detected changes?
Terraform will write these changes to the state without modifying any real
infrastructure.
```

평소 **apply** 의 문구는 **Do you want to perform these actions?** 였습니다. 두 문장을 나란히 놓고 보세요. 하나는 실물에 손을 대겠다는 것이고, 하나는 state 에만 쓰겠다는 것입니다.

이제 state 를 다시 열어봅니다.

```
# pwd == practice
terraform state show aws_s3_bucket.drift_demo
```

```
"Env"    = "HACKED"
```

A-4에서는 **study** 였던 자리가 **HACKED** 로 바뀌었습니다. **AWS** 에는 아무 요청도 보내지 않았고 state 만 고쳤습니다.

그런데 **plan** 을 다시 쳐보세요.

```
# pwd == practice
terraform plan
```

```
Plan: 0 to add, 1 to change, 0 to destroy.
```

여전히 **1 to change** 입니다. state 와 실물은 같아졌지만 코드는 그대로 **study** 이기 때문입니다. **-refresh-only** 는 어긋남을 없애는 명령이 아니라, 셋 중 둘을 먼저 맞춰서 남은 차이가 코드와 실물 사이의 것임을 분명하게 만드는 명령입니다. 개념워크북 8번.

이제 **실물을 코드에 맞추는** 쪽입니다.

```
# pwd == practice
terraform apply    # yes
terraform plan
```

```
Apply complete! Resources: 0 added, 1 changed, 0 destroyed.
```

```
No changes. Your infrastructure matches the configuration.
```

셋이 다시 같아졌습니다. 콘솔에서 확인해 보면 **Env** 태그가 **study** 로 돌아와 있습니다.



팁

콘솔 변경이 옳고 코드가 낡은 경우라면 `-refresh-only` 뒤에 코드를 직접 수정해야 합니다. `-refresh-only` 만 하고 끝내면 다음 사람이 `apply` 하는 순간 실물이 코드 쪽으로 되돌아갑니다. 개념워크북 9번의 표를 보세요.

A-6 기록

- `-refresh-only` 실행 후 `state show` 의 `Env` : _____
- `-refresh-only` 실행 후 `plan` 의 마지막 줄: `Plan: ____ to add, ____ to change, ____ to destroy.`
- 그 숫자가 0이 아닌 이유를 한 문장으로: _____
- `apply` 실행 후 콘솔의 `Env` 값: _____
- `apply` 와 `apply -refresh-only` 가 각각 무엇을 무엇에 맞추는지: _____

Block B. 각자 진행 (20분): 콘솔이 먼저 만들어 놓습니다

강사는 순회합니다. 막히면 `solution/` 을 보되, 먼저 오류 메시지의 첫 줄을 읽어보세요.

[실습] B-1. 콘솔에서 버킷을 직접 만듭니다 (3분)

지금 하는 일은 Terraform 이 모르는 리소스를 일부러 만드는 것입니다.

AWS 콘솔에서 S3 로 갑니다.

1. **Create bucket** 을 누릅니다
2. 이름에 `{project_name}-manual` 을 넣습니다. 예: `boaz26-w6-kdh1834-manual`
3. 리전이 **아시아 태평양(서울) ap-northeast-2** 인지 확인합니다
4. **Bucket Versioning** 을 **Enable** 로 바꿉니다
5. **Block Public Access** 는 기본값 그대로 둡니다. 네 항목이 모두 체크되어 있어야 합니다
6. **Tags** 는 하나도 넣지 마세요
7. 나머지도 기본값 그대로 두고 아래로 내려 **Create bucket** 을 누릅니다

4번을 꼭 하세요. B-3에서 이 설정이 필요합니다.

⚠ 주의

6번을 지키세요. 콘솔에서 태그를 붙이면 B-3의 `plan` 이 `No changes` 가 아니라 "태그를 지우겠다"는 diff 로 나옵니다. 코드에 `tags` 를 안 적었기 때문입니다. 그 자체로 틀린 것은 아니지만 오늘 확인하려는 내용이 드러나지 않습니다. 이미 붙였다면 콘솔에서 태그를 지우고 오세요.

⚠ 주의

이름이 이미 쓰이고 있다는 오류가 나면 뒤에 숫자를 붙이세요. 그리고 `terraform.tfvars` 의 `project_name` 은 그대로 두고, 코드에서 이 버킷을 가리키는 이름만 맞추면 됩니다. 헛갈리면 강사에게 물어보세요.

터미널로 돌아와서 Terraform 이 이 버킷을 아는지 봅니다.

```
# pwd == practice
terraform plan
```

```
No changes. Your infrastructure matches the configuration.
```

방금 버킷을 만들었는데 `plan` 은 아무 말이 없습니다. `state` 에 없는 것은 Terraform 에게 없는 것과 같습니다. 개념워크북 10번.

[실습] [함정] B-2. TODO ③: 빈 리소스 블록을 작성하고 `import` 합니다 (4분)

지금 하는 일은 이미 있는 버킷을 `state` 에 편입하는 것입니다.

`practice/main.tf` 의 TODO ③ 자리에 블록을 적습니다. 속성은 `bucket` 한 줄뿐입니다.

```
resource "aws_s3_bucket" "manual" {
  bucket = "${var.project_name}-manual"
}
```

여기서 `apply` 를 치면 안 됩니다. 왜 안 되는지 `plan` 으로 먼저 확인하세요.

```
# pwd == practice
terraform plan
```

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

Terraform 은 `state` 에 이 리소스가 없으니 새로 만들어야 한다고 판단합니다. AWS 에 그 이름의 버킷이 이미 있다는 것을 모릅니다. 필요한 것은 "만들어라"가 아니라 "이미 있는 저것이 이 코드가 가리키는 것이다"라고 알려주는 절차입니다.

`import` 하기 전에 `state` 를 한 번 봅니다. 이미 있는데 또 `import` 하면 오류가 나기 때문입니다.

```
# pwd == practice
terraform state list
terraform import aws_s3_bucket.manual boaz26-w6-kdh1834-manual
```

버킷 이름 자리에는 본인 값을 넣으세요. `{project_name}` 은 셸이 치환해주지 않습니다.

```
aws_s3_bucket.manual: Importing from ID "boaz26-w6-kdh1834-manual"...
aws_s3_bucket.manual: Import prepared!
  Prepared aws_s3_bucket for import
aws_s3_bucket.manual: Refreshing state... [id=boaz26-w6-kdh1834-manual]

Import successful!
```

The resources that were imported are shown above. These resources are now in your Terraform state and will henceforth be managed by Terraform.

```
# pwd == practice
terraform state list
```

```
aws_s3_bucket.drift_demo
aws_s3_bucket.manual
```

2줄이 되었습니다.

[함정] 코드에 블록을 안 적고 import 를 먼저 치면 이렇게 막힙니다.

```
Error: resource address "aws_s3_bucket.manual" does not exist in the configuration.

Before importing this resource, please create its configuration in the root module. For example:

resource "aws_s3_bucket" "manual" {
  # (resource arguments)
}
```

마지막 줄의 `# (resource arguments)` 를 보세요. 인자 자리가 비어 있습니다. Terraform 이 채워주는 것은 state 뿐이고 괄호 안은 사람이 씁니다. 오늘의 주제가 이 한 줄에 들어 있습니다. 개념워크북 11번.

[함정] 같은 주소로 두 번 import 하면 이렇게 막힙니다.

```
Error: Resource already managed by Terraform

Terraform is already managing a remote object for aws_s3_bucket.manual. To
import to this address you must first remove the existing object from the
state.
```

`terraform state rm aws_s3_bucket.manual` 로 state 에서 뺀 뒤 다시 import 하라는 뜻입니다. `state rm` 은 state 에서만 지우고 실물은 그대로 둡니다.

[함정] 없는 버킷을 import 하려 하면 리전과 이름을 확인하세요. 콘솔의 리전이 서울이 아니면 여기서 걸립니다.

[확인] [관찰 🍷] B-3. No changes 가 무엇을 뜻하는지 (3분)

지금 하는 일은 import 직후의 `plan` 을 읽고, 그것이 무엇을 말하지 않는지 확인하는 것입니다.

```
# pwd == practice
terraform plan
```

```
No changes. Your infrastructure matches the configuration.
```

다 된 것처럼 보입니다. 그런데 코드를 다시 보세요.

```
resource "aws_s3_bucket" "manual" {
  bucket = "boaz26-w6-kdh1834-manual"
}
```

B-1에서 **버전 관리를 했습니다**. 그런데 코드에는 그 이야기가 한 글자도 없습니다. 그런데도 **plan** 은 아무 말이 없습니다.

plan 이 비교하는 것은 **코드에 적힌 것과 실물의 대응하는 부분**입니다. 코드에 없는 설정은 비교 대상이 아니고, 비교 대상이 아니면 차이도 없습니다. **No changes** 는 "코드에 적힌 범위 안에서는 어긋난 것이 없습니다"라는 뜻입니다. 개념워크북 13번.

🔗 참고

모든 속성이 이렇게 아무 변경도 표시하지 않는 것은 아닙니다. **tags** 는 코드에 안 적으면 "실물의 태그를 지우겠다"는 diff 로 나옵니다. B-1에서 콘솔 태그를 넣지 말라고 한 이유입니다. 속성마다 "코드에 없으면 무시" 와 "코드에 없으면 비우겠다" 중 어느 쪽인지가 provider 스키마에 정해져 있고, 문서만 봐서는 구분되지 않습니다. 실물이 있는 리소스를 코드로 옮길 때 **plan** 을 한 번 돌려보는 것 말고는 확인할 방법이 없다는 것도 고전 import 의 불편 중 하나입니다.

실물이 실제로 어떤지는 state 로 확인할 수 있습니다.

```
# pwd == practice
terraform state show aws_s3_bucket.manual
```

state 에는 import 가 읽어온 값들이 들어 있습니다. 코드에 적힌 것보다 훨씬 많습니다. 이 차이가 다음 스텝에서 할 일입니다.

B-3 기록

- import 직후 **plan** 의 결과: _____
- 콘솔에서 컨 설정 중 코드에 없는 것: _____
- No changes** 가 "코드가 실물을 다 담았다"는 뜻이 아닌 이유를 한 문장으로: _____

[실습] [함정] B-4. TODO ④: 부속 리소스를 코드로 가져옵니다 (3분)

지금 하는 일은 콘솔에서 컨 설정을 코드에 적고, 그것이 왜 곧바로 맞아떨어지지 않는지 보는 것입니다.

S3 버킷의 설정들은 **aws_s3_bucket** 블록 안에 있지 않고 **별도 리소스로 분리**되어 있습니다. AWS provider 4.x 부터 그렇습니다. 개념워크북 14번.

practice/main.tf 의 TODO ④ 자리에 블록 두 개를 적습니다.

```
resource "aws_s3_bucket_versioning" "manual" {
  bucket = aws_s3_bucket.manual.id

  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_public_access_block" "manual" {
```

```

bucket          = aws_s3_bucket.manual.id
block_public_acls = true
block_public_policy = true
ignore_public_acls = true
restrict_public_buckets = true
}

```

아래 블록의 네 값이 모두 `true` 인 것은 콘솔에서 버킷을 기본값으로 만들면 퍼블릭 접근이 전부 차단되기 때문입니다.

```

# pwd == practice
terraform plan

```

```

Plan: 2 to add, 0 to change, 0 to destroy.

```

🚫 절대 금지

여기서 **apply** 를 **치지 마세요**. 실물에는 이미 버전 관리가 켜져 있고 퍼블릭 접근 차단도 걸려 있습니다. 그런데 state 에 없으니 Terraform 은 새로 만들겠다고 합니다. B-2의 상황이 두 번 더 반복된 것입니다.

S3 의 이 두 설정은 다시 걸어도 결과가 같지만, 리소스 종류에 따라서는 apply 가 기존 설정을 덮어써서 되돌릴 수 없는 것도 있습니다. **실물이 이미 있으면 apply 가 아니라 import 입니다**. 개념워크북 14번의 IMPORTANT.

[실습] [확인] [관찰 🍷] B-5. 부속 리소스를 각각 import 합니다 (4분)

지금 하는 일은 리소스 개수만큼 import 를 반복하는 것입니다. 이 반복이 오늘 겪어야 할 불편입니다.

두 리소스의 import ID 는 셋 다 버킷 이름입니다. 리소스마다 형식이 다르므로 provider 문서의 Import 절에서 확인하는 것이 원칙입니다. 개념워크북 12번.

```

# pwd == practice
terraform import aws_s3_bucket_versioning.manual boaz26-w6-kdh1834-manual
terraform import aws_s3_bucket_public_access_block.manual boaz26-w6-kdh1834-manual

```

각각 **Import successful!** 이 나와야 합니다.

```

# pwd == practice
terraform plan

```

```

No changes. Your infrastructure matches the configuration.

```

이제 버전 관리가 코드에도 state 에도 실물에도 있습니다. B-3의 **No changes** 와 지금의 **No changes** 는 화면이 같지만 뜻이 다릅니다. 그때는 코드가 비어 있어서 비교할 것이 없었고, 지금은 코드에 적힌 것이 실물과 맞습니다.

```

# pwd == practice
terraform state list | tee ~/week6-state-list.txt

```

```
aws_s3_bucket.drift_demo
aws_s3_bucket.manual
aws_s3_bucket_public_access_block.manual
aws_s3_bucket_versioning.manual
```

4줄입니다. `tee` 로 홈 디렉터리에 사본을 남기는 것에 주의하세요. C-1 에서 `destroy` 를 하면 `state list` 가 비어서 다시 출력할 수 없고, C-3 의 마스킹과 제출물이 이 파일을 씁니다. 리포 안이 아니라 `~` 에 두는 것은 마스킹 전에 실수로 커밋되지 않게 하기 위해서입니다.

S3 객체도 확인합니다. 지난 주차들의 state 가 같은 버킷에 남아 있는지 함께 보세요.

```
# pwd == practice
aws s3 ls --recursive "s3://boaz26-w3-{본인-github-id}-tfstate/"
```

`week06/app/terraform.tfstate` 가 새로 생겼고 `week03` · `week04` · `week05` 것이 그대로 있어야 합니다. `key` 가 다르면 서로 다른 state 입니다.

B-5 기록

- `terraform import` 를 오늘 몇 번 실행했는가: _____
- `state list` 4줄 중 `apply` 로 만든 것은 몇 개인가: _____
- `state list` 4줄 중 `import` 로 편입한 것은 몇 개인가: _____
- 콘솔에서 버킷 하나를 만들었는데 Terraform 리소스가 셋이 된 이유: _____
- 리소스가 10개인 인프라를 이 방식으로 옮긴다면 `import` 를 몇 번 쳐야 하는가: _____
- 버킷에 있는 `.tfstate` 객체 `key` 를 전부 적기: _____

[실습] B-6. TODO ⑤: output 을 채우고 마무리 검사 (2분)

지금 하는 일은 코드를 정리하고 제출 전 검사를 도는 것입니다.

`practice/outputs.tf` 의 TODO ⑤ 를 채웁니다.

```
output "manual_bucket" {
  description = "콘솔이 만들고 import 로 편입한 버킷의 이름"
  value       = aws_s3_bucket.manual.id
}
```

```
# pwd == practice
terraform fmt
terraform validate
terraform plan
terraform output
```

Success! The configuration is valid.

```
No changes. Your infrastructure matches the configuration.
```

```
drift_bucket = "boaz26-w6-kdh1834-drift"
manual_bucket = "boaz26-w6-kdh1834-manual"
```

[함정] TODO ③ 을 채우기 전에 이 output 을 먼저 적으면 아직 없는 리소스를 가리켜서 `validate` 가 실패합니다.

```
Error: Reference to undeclared resource
```

```
on outputs.tf line 2, in output "manual_bucket":
  2:   value = aws_s3_bucket.manual.id
```

```
A managed resource "aws_s3_bucket" "manual" has not been declared in the
root module.
```

[실습] [관찰 🏹] B-7 (심화 ✨). `state rm` 과 다음 주 맛보기

먼저 끝낸 사람만 하세요. 셋 중 하나만 골라도 됩니다.

첫째, `state rm` 으로 관리에서 빼봅니다. 실물은 그대로 있고 state 에서만 빠집니다.

```
# pwd == practice
terraform state rm aws_s3_bucket_versioning.manual
terraform state list      # 3줄로 줄어듭니다
terraform plan            # 1 to add. B-4 의 상황으로 되돌아갔습니다
terraform import aws_s3_bucket_versioning.manual boaz26-w6-kdh1834-manual
terraform plan            # 다시 No changes
```

콘솔에서 버킷을 확인해 보세요. `state rm` 을 했을 때 버전 관리가 꺼지지 않았습니다. state 에서 뺀 것과 실물을 지운 것은 다릅니다.

둘째, 모듈 안 리소스의 `import` 주소를 확인합니다. 실행하지 말고 형태만 보세요.

```
terraform import 'module.network.aws_vpc.this' vpc-0abc123
terraform import 'module.network.aws_subnet.public["ap-northeast-2a"]' subnet-0abc123
```

작은따옴표가 필요한 이유는 week4·week5에서 `state show` 를 쓸 때와 같습니다. 대괄호와 큰따옴표를 셸이 먼저 해석합니다. `for_each` 로 만든 리소스는 인스턴스 하나하나를 따로 import 해야 합니다. 개념워크북 15번.

셋째, 다음 주에 무엇이 달라지는지 봅니다. 아래는 7주차에 쓸 문법입니다. 오늘은 실행하지 마세요.

```
import {
  to = aws_s3_bucket.manual
  id = "boaz26-w6-kdh1834-manual"
}
```

명령이 아니라 코드입니다. 코드라서 PR 리뷰를 받고 git 히스토리에 남습니다. 여기에 `terraform plan -generate-config-out=generated.tf` 를 더하면 HCL 초안까지 만들어 줍니다. 오늘 직접 옮겨 적은 부분을 이 기능이 대신합니다.

B-7 기록

- `state rm` 후 콘솔에서 버전 관리가 꺼졌는가: _____
- `state rm` 과 실물 삭제의 차이를 한 문장으로: _____
- 오늘 방식과 `import` 블록 방식의 차이를 한 문장으로: _____

Block C. 정리 (7분): destroy 와 잔존 점검

🚫 절대 금지

이 블록을 빼면 S3 버킷 두 개가 남습니다. 요금은 거의 없지만 버킷 이름이 전 세계에서 유일해서 다음에 같은 이름을 쓸 수 없습니다. 그리고 import 한 것도 지워진다는 것을 확인하는 것이 이번 주 마무리입니다.

[확인] C-1. destroy 하고 무엇이 지워지는지 봅니다 (3분)

지금 하는 일은 import 가 무엇을 넘겼는지 확인하는 것입니다.

`destroy` 를 치기 전에 `plan` 으로 먼저 세어 보세요.

```
# pwd == practice
terraform plan -destroy
```

Plan: 0 to add, 0 to change, 4 to destroy.

4개입니다. 그중 `manual` 쪽 셋은 내가 콘솔에서 직접 만든 것입니다. Terraform 이 만든 적이 없습니다. 그런데 import 를 한 순간부터 Terraform 의 관리 대상이 되었고, 관리 대상은 삭제 대상이기도 합니다. 개념워크북 17번.

```
# pwd == practice
terraform destroy      # yes
```

Destroy complete! Resources: 4 destroyed.

[함정] `BucketNotEmpty` 로 실패하면 버킷 안에 객체가 있는 것입니다. 콘솔에서 파일을 지우거나 아래 명령으로 비운 뒤 다시 실행하세요.

```
aws s3 rm "s3://boaz26-w6-kdh1834-manual" --recursive
```

[확인] [관찰 🕒] C-2. state 와 계정을 따로 확인 (2분)

`state list` 가 비었다는 것은 Terraform 이 관리하지 않는다는 뜻일 뿐, 계정에 아무것도 없다는 뜻이 아닙니다. 오늘 그 차이에 이름을 붙였습니다. 개념워크북 3번.

```
# pwd == practice
terraform state list      # 빈 출력이어야 합니다
```

계정 쪽은 버킷 목록으로 확인합니다.

```
# pwd == practice
aws s3 ls | grep -E "w6|drift|manual"
```

빈 출력이어야 합니다. 콘솔에서 S3 목록을 직접 봐도 됩니다.

week3의 저장소는 남아 있어야 정상입니다.

```
# pwd == practice
aws s3 ls | grep tfstate
aws dynamodb list-tables --region ap-northeast-2 --query 'TableNames' --output text
```

무엇	오늘 세션 끝에
<code>{project_name}-drift</code> · <code>{project_name}-manual</code> 버킷	0개여야 합니다
week3의 S3 버킷(<code>*-tfstate</code>)과 DynamoDB 테이블(<code>*-tflock</code>)	남아 있어야 정상입니다
버킷 안의 <code>week03</code> ~ <code>week06</code> state 객체	남아 있어야 정상입니다

week3의 state 저장소는 7주차까지 남깁니다. 반대로 하면 state 저장소가 먼저 사라져서 오늘 만든 것을 지울 수단이 없어집니다.

C-2 기록

- `terraform state list` 가 비었는가: _____
- 계정에 `-drift` · `-manual` 버킷이 남아 있는가: _____
- week3의 S3 버킷과 DynamoDB 테이블은 남아 있는가: _____
- 콘솔에서 만든 버킷이 `destroy` 로 지워진 이유를 한 문장으로: _____

[실습] C-3. 제출 전 마스킹 (1분)

지금 하는 일은 증빙에서 가려야 할 값을 지우는 것입니다.

```
# pwd == week6-import-drift
cd ..
```

```
ID=본인-github-id      # 예시 값 그대로 두지 마세요
echo "$ID"              # 바꿨는지 눈으로 확인
mkdir -p "submissions/$ID"
```

B-5에서 `~/week6-state-list.txt` 로 남겨둔 사본을 가져옵니다. C-1에서 `destroy` 를 했으므로 지금 `state list` 를 다시 출력하면 빈 파일이 됩니다.

계정번호 12자리를 가립니다. macOS 와 GNU 에서 `sed -i` 문법이 다르므로 파일을 새로 만드는 쪽이 안전합니다.

```
# pwd == week6-import-drift
sed -E \
-e 's/[0-9]{12}/<account-id>/g' \
-e 's/([0-9]{1,3}\.){3}[0-9]{1,3}/x.x.x.x/g' \
~/week6-state-list.txt > "submissions/$ID/state-list.txt"
```

마스킹이 실제로 됐는지 파일을 눈으로 확인합니다. 4줄이 그대로 있어야 합니다.

```
# pwd == week6-import-drift
cat "submissions/$ID/state-list.txt"
grep -nE '([0-9]{1,3}\.){3}[0-9]{1,3}|[0-9]{12}' "submissions/$ID/state-list.txt"
```

두 번째 명령은 아무것도 안 나와야 정상입니다.

A-5의 `plan` 화면도 제출물입니다. 터미널에서 복사해 `submissions/$ID/plan-drift.txt` 로 저장하고 같은 방식으로 마스킹하세요.

🚫 절대 금지

스크린샷도 마스킹하세요. 명령으로 걸러지지 않습니다. 가려야 하는 것은 계정번호 12자리와 액세스 키입니다. 이번 주는 공인 IP 를 쓰지 않지만, 지난 주차 화면이 섞여 있다면 함께 가리세요.

[개념] C-4. week3 저장소는 그대로 둡니다 (1분)

읽고 넘어가세요. week3에서 만든 S3 버킷과 DynamoDB 테이블은 7주차까지 남기고, 7주차도 같은 버킷에 `key` 만 바꿔 씁니다. 지우는 절차는 week3 실습워크북 C-5에 있고 이번 주에는 실행하지 않습니다.

세션 후: 실습 결과 제출 (브랜치 → PR)

세션 중에는 `practice/` 에서 실습하면 됩니다. 세션이 끝난 뒤 아래 절차를 진행하세요.

```
# pwd == week6-import-drift
ID=본인-github-id      # {github-id} 는 셸이 치환해주지 않습니다

git switch main && git pull
git switch -c "week6/$ID"
```

```
mkdir -p "submissions/$ID"
cp practice/*.tf "submissions/$ID/"
cp practice/example.tfvars "submissions/$ID/"
# C-3에서 저장한 state-list.txt 와 plan-drift.txt 는 이미 그 폴더에 있습니다 (마스킹 확인!)
# 관찰 🍷 답안을 submissions/$ID/observations.md 로 정리

git add "submissions/$ID"

# 스테이징한 내용에서 계정번호 · 액세스 키 · 공인 IP 를 찾습니다. 빈 출력이어야 합니다.
git diff --cached | grep -nE '([0-9]{1,3}\.){3}[0-9]{1,3}|[0-9]{12}|AKIA[0-9A-Z]{16}'
git status --porcelain | grep -E 'terraform\.tfvars|\.tfstate'
git status # tfvars / tfstate / .terraform 안 올라갔는지!
git commit -m "week6: $ID 실습 제출"
git push -u origin "week6/$ID"
```

넣을 것

넣지 말 것

practice/ 의 .tf 파일 전부 (backend.tf 포함)

terraform.tfvars

example.tfvars (값은 채우지 않은 상태)

terraform.tfstate · *.backup

state-list.txt (마스킹 완료. B-5의 4줄)

.terraform/ 폴더

plan-drift.txt (마스킹 완료. A-5의 화면)

계정번호 12자리 · 액세스 키

observations.md (관찰 🍷 답안)

-

복사가 끝나면 practice/ 는 되돌려 둡니다.

```
# pwd == week6-import-drift
git restore practice/ # 복사를 끝낸 뒤에만! 스켈레톤으로 되돌아갑니다
```

그다음 GitHub에서 week6/{github-id} 에서 main 으로 PR을 올리면 강사가 리뷰 후 머지합니다.

! 꼭 기억

제출은 submissions/{github-id}/ 본인 폴더에 하세요. practice/ 를 직접 고쳐서 올리면 머지하는 순간 다음 사람이 풀 # TODO 가 사라집니다.

과제④는 이 리포가 아니라 assignments 리포의 round4-week6/{github-id}/ 에 넣니다. 명세는 assignment/ASSIGNMENT.md 에 있습니다.

체크포인트 (Definition of Done)

- [] A-1: week3 버킷과 테이블이 살아 있는 것 확인. 콘솔 리전이 서울
- [] A-2: backend.tf 의 key 가 week06/app/terraform.tfstate
- [] A-3: plan 이 Plan: 1 to add, 0 to change, 0 to destroy. 이고 apply 성공
- [] A-4: 콘솔에서 Env 태그를 바꾼 뒤 state show 에는 아직 옛날 값

- [] A-5: `plan` 이 `Plan: 0 to add, 1 to change, 0 to destroy.` 로 드리프트 감지
- [] A-5: `plan` 화면의 두 덩어리를 구분해서 읽었습니다
- [] A-6: `apply -refresh-only` 후 state 가 `HACKED` 로 바뀌었고, `plan` 은 여전히 `1 to change`
- [] A-6: `apply` 후 `No changes.` 이고 콘솔에서 태그가 `study` 로 복원
- [] B-1: 콘솔에서 `-manual` 버킷 생성. 버전 관리 Enable
- [] B-2: `Import successful!` 확인. `state list` 2줄
- [] B-3: `No changes` 인데도 버전 관리가 코드에 없다는 것을 확인
- [] B-4: 부속 리소스 블록 두 개를 적고 `plan` 이 `2 to add`. `apply` 하지 않았습니다
- [] B-5: 부속 두 개 import 후 `plan` 이 `No changes.` 이고 `state list` 4줄
- [] B-5: S3 에 `week06/app/terraform.tfstate` 객체 1개
- [] B-6: `validate` 성공. `terraform output` 에 버킷 두 개
- [] C-1: `plan -destroy` 가 `4 to destroy` 이고 `destroy` 완료 (`4 destroyed`)
- [] C-2: `state list` 빈 출력. 계정도 두 버킷이 없음
- [] C-3: 마스킹 확인. 계정번호가 증빙에 없습니다
- [] week3의 S3와 DynamoDB는 지우지 않았습니다

오늘의 명령 요약

```
# Block A : 콘솔이 나중에 끼어듭니다
cd practice
cp example.tfvars terraform.tfvars      # project_name
# backend.tf 의 CHANGE-ME 두 곳을 week3 값으로 (key 는 건드리지 않기)
# TODO ② main.tf 에 aws_s3_bucket.drift_demo
terraform init
terraform plan                          # Plan: 1 to add   <- 아니면 멈춤
terraform apply                         # yes
terraform state list                    # 1줄
# 콘솔에서 Env 태그를 study -> HACKED
terraform state show aws_s3_bucket.drift_demo # 아직 study
terraform plan                          # Plan: 0 to add, 1 to change, 0 to destroy.
terraform apply -refresh-only           # yes. state 만 실물에 맞춥니다
terraform state show aws_s3_bucket.drift_demo # HACKED 로 바뀜
terraform plan                          # 여전히 1 to change (코드는 그대로라서)
terraform apply                         # yes. 실물을 코드에 맞춥니다
terraform plan                          # No changes.

# Block B : 콘솔이 먼저 만들어 놓습니다
# 콘솔에서 {project_name}-manual 버킷 생성 (버전 관리 Enable)
terraform plan                          # No changes. <- Terraform 은 모릅니다
# TODO ③ main.tf 에 aws_s3_bucket.manual 빈 블록
terraform plan                          # Plan: 1 to add   <- apply 하지 말 것
terraform state list                    # import 전에 확인하는 습관
terraform import aws_s3_bucket.manual <버킷명>
terraform plan                          # No changes. <- 그런데 버전 관리는 코드에 없습니다
# TODO ④ main.tf 에 versioning · public_access_block
terraform plan                          # Plan: 2 to add   <- apply 하지 말 것
terraform import aws_s3_bucket_versioning.manual <버킷명>
terraform import aws_s3_bucket_public_access_block.manual <버킷명>
```

```

terraform plan                                # No changes.
terraform state list | tee ~/week6-state-list.txt      # 4줄
# TODO ⑤ outputs.tf
terraform fmt && terraform validate && terraform output

# Block C : 정리
terraform plan -destroy                        # 4 to destroy
terraform destroy                             # yes. 4 destroyed
terraform state list                          # 빈 출력
aws s3 ls | grep -E "w6|drift|manual"         # 빈 출력

```

다음 주 예고

Week 7. 최신 import 와 GitHub Actions [대면]. 오늘 직접 반복한 작업을 다음 주에는 코드로 처리합니다.

오늘 본 것	Week7에서 다루는 지점
<code>terraform import</code> 를 리소스 개수만큼 한 번씩 쳤습니다	<code>import</code> 블록을 코드에 적어두면 <code>apply</code> 한 번으로 여러 개를 편입합니다
<code>state show</code> 를 보고 속성을 직접 옮겨 적었습니다	<code>terraform plan -generate-config-out=파일.tf</code> 가 HCL 초안을 만들어 줍니다
<code>import</code> 명령은 실행 기록이 코드에 남지 않습니다	<code>import</code> 블록은 코드라서 PR 리뷰를 받고 git 히스토리에 남습니다
명령을 본인 노트북에서 쳤습니다	GitHub Actions 에서 <code>plan</code> 을 돌려 PR 에 붙입니다
콘솔 변경을 사람이 눈으로 발견했습니다	정기적으로 <code>plan</code> 을 돌려 드리프트를 감지하는 방법을 봅니다

예습: `import` 블록 문법, `-generate-config-out` 옵션, GitHub Actions 기본(워크플로 파일 · job · step)

부록. 트러블슈팅

증상	원인	해결
<code>resource address "..."</code> does not exist in the configuration	코드에 블록을 안 적고 import 함	먼저 <code>resource</code> 블록을 적으세요. B-2
Resource already managed by Terraform	같은 주소로 중복 import	<code>terraform state list</code> 로 확인. 필요하면 <code>state rm</code> 후 재시도. B-2
import 가 "없는 것"이라며 실패	버킷 이름 오타 또는 리전 불일치	콘솔 리전이 서울인지, 이름이 정확한지 확인. <code>aws s3api head-bucket --bucket <이름></code>
import 후에도 diff 가 남음	코드에 속성이 다 안 채워짐	<code>terraform state show <주소></code> 로 실물을 보고 하나씩 옮겨 적기. B-4

증상	원인	해결
<code>plan</code> 이 <code>No changes</code> 인데 콘솔 설정이 코드에 없음	그 설정이 코드에 없어서 비교 대상이 아님	부속 리소스 블록을 적고 각각 import. B-3 · B-4
<code>state show</code> 와 콘솔 값이 다름	state 는 자동으로 갱신되지 않음	정상입니다. <code>plan</code> 이나 <code>apply -refresh-only</code> 가 refresh 합니다. A-4
<code>-refresh-only</code> 후에도 <code>plan</code> 이 <code>1 to change</code>	코드가 그대로라서	정상입니다. 코드를 고치거나 <code>apply</code> 하세요. A-6
S3 버킷 이름이 이미 쓰인다는 오류	버킷 이름은 전 세계에서 유일	<code>project_name</code> 에 본인 github id 를 넣고 뒤에 숫자를 붙이기
<code>Invalid value for variable</code>	<code>project_name</code> 에 대문자나 밑줄	소문자 · 숫자 · 하이픈만. A-2
<code>Reference to undeclared resource</code>	output 이 아직 없는 리소스를 가리킴	TODO 순서대로 진행. B-6
<code>destroy</code> 가 <code>BucketNotEmpty</code> 로 실패	버킷에 객체가 있음	<code>aws s3 rm s3://<버킷> --recursive</code> 후 재시도. C-1
<code>Failed to get existing workspaces: S3 bucket ... does not exist</code>	week3 버킷 이름 오타 또는 bootstrap 없음	A-1로 돌아가기
<code>Error acquiring the state lock</code>	동시 실행 또는 비정상 종료	아무도 작업 안 하는 것 확인 후 <code>terraform force-unlock <LOCK_ID></code>
<code>Warning: Deprecated Parameter</code>	<code>dynamodb_table</code> 사용	정상입니다. week3 개념워크북 9번
<code>zsh: no matches found</code>	주소의 대괄호를 셸이 해석	작은따옴표로 감싸기. B-7
<code>plan</code> 숫자가 표와 전혀 다름	<code>backend.tf</code> 의 <code>key</code> 가 지난 주 것	apply 하지 말고 <code>key</code> 를 <code>week06/app/...</code> 로. A-2

참고 문서

- [Import 개요](#) : B-2
- [Import 사용법](#) : B-2 · B-5
- [Manage Resource Drift 튜토리얼](#) : Block A 전체
- [terraform refresh 와 -refresh-only](#) : A-6
- [aws_s3_bucket_versioning](#) : B-4. 문서 맨 아래 Import 절
- [aws_s3_bucket_public_access_block](#) : B-4
- [terraform state 하위 명령](#) : A-4 의 `state show` , B-7 의 `state rm`

- [import 블록](#) : B-7. 7주차 예습
- week3 실습워크북 B-4 · B-5 : backend 로 state 를 옮기는 절차
- week5 실습워크북 C-2 : state 와 계정을 따로 확인하던 자리

이 문서에 실린 오류 메시지 네 종류(`resource address ... does not exist in the configuration` · `Resource already managed by Terraform` · `Reference to undeclared resource` · `Invalid value for variable`)와 `terraform state list` · `terraform state show` 의 출력 모양은 [solution/](#) 코드를 Terraform 1.15.8 · `darwin_arm64` 에서 실제로 실행해 얻은 값입니다. AWS 자격증명 없이 재현되므로 과금이 없습니다.

세 리소스의 import ID 가 모두 버킷 이름이라는 것은 AWS provider 문서(`aws_s3_bucket` · `aws_s3_bucket_versioning` · `aws_s3_bucket_public_access_block`)의 Import 절에서 확인했습니다. `aws_s3_bucket` 을 `bucket` 한 줄만 적고 import 했을 때 버전 관리가 diff 로 잡히지 않는 것과 태그는 diff 로 잡히는 것은 provider 스키마를 직접 조회해 확인했습니다. 버킷 리소스의 `versioning` 블록은 `Optional` 이면서 `Computed` 라 코드에 없으면 state 값을 그대로 두고, `tags` 는 `Computed` 가 아니라서 코드에 없으면 비우려 합니다.

이 문서가 인용한 Terraform 안내 문구는 같은 버전 바이너리의 메시지 목록에서 원문을 대조했습니다. `Note: Objects have changed outside of Terraform` 과 그 아래 설명문, `has changed` · `will be updated in-place` · `forces replacement` , `No changes. Your infrastructure matches the configuration.` , `Import successful!` 과 `... will henceforth be managed by Terraform.` , A-6 의 `-refresh-only` 승인 프롬프트 두 줄, 평소 `apply` 의 `Do you want to perform these actions?` , `Destroy complete! Resources:` 가 여기에 해당합니다. 문구는 원문이고, 그 문구가 실제로 이 실습에서 뜨는지는 `apply` 로 확인하지 않았습니다.

확인하지 않은 것은 숫자와 값입니다. `plan` 마지막 줄의 `1 to add` · `1 to change` · `2 to add` , `state list` 4줄, `4 destroyed` 는 리소스 정의에서 센 값입니다. 태그 diff 에 찍히는 값, `terraform output` 의 값 모양, `aws s3 ls` 출력도 실제로 실행해 얻은 것이 아닙니다. 비용은 개념워크북 Part 6과 같습니다.

! 꼭 기억

끝났으면 `destroy` 를 했는지, 계정에 `-drift` 와 `-manual` 버킷이 남아 있지 않은지, 그리고 week3의 S3와 DynamoDB는 건드리지 않았는지 한 번 더 확인하세요. 버킷을 남겨두면 그 이름을 다시 쓸 수 없고, week3 버킷을 지우면 7주차부터 처음부터 다시 만들어야 합니다.